NERVE: Software Without Code

Compiling Behavior Specifications into Exact Weights and Training the Rest as an Energy

Alex Welcing
Lupine Science, Inc.
alexwelcing@gmail.com

Abstract

We ask what it takes to ship a small interactive program as a neural network that is held to its specification, rather than as code. NERVE takes a behavior spec—typed state, guarded pure transitions, invariants, post-conditions, the set of states a gesture may lead to, and *holes* given only by examples—and produces a network that maps a state and a user action to the next state by minimizing an energy over candidate futures. The energy has three terms. Transitions the spec writes down are compiled, with no training, into a two-layer ReLU network whose energy is exactly the normalized Hamming distance to the true successor; it is silent on holes. An action-conditioned joint-embedding predictive architecture supplies prediction error as energy. A factored per-field head with a shift kernel indexed by the offset from the field's current value makes "+1" the same function at every value of the field. Trained with InfoNCE over all candidates and a hinge on spec-violating futures, the learned terms alone—the circuit disabled—match the oracle on every held-out (state, action) cell for seven finite interfaces of up to 404 states $\times$ 103 actions in the repository's runs, three of the interfaces specified from screenshots by a vision agent without hand edits; three-seed CPU re-runs of the four hand-written specs reproduce this on negotiation and the gridded slider and miss at most one boundary cell on the counter. A continuous slider reaches 99.8% within tolerance by gradient descent on the energy in the repository's run (91.5% in our CPU re-run from the default log). Ablations locate the factored head's contribution: where a rule is a shift along an ordinal field it carries the held-out cells as the held-out fraction grows (slider with 30% of cells held out: 436 of 449 without it, 448 with it; counter: 67 and 71 of 76), while the negotiation game, whose rules are absolute assignments, is recovered exactly from half of its transition table by the JEPA term alone. Conformance is measured on the BF16 weights that ship; a Cloudflare Worker and a browser execute the same network; continued training is shipped as bit-identical sparse weight patches; and an evolving team of coding agents produced the negotiation network for $14.23. We also say plainly what this is not: the interfaces are small, the compiled circuit is a lookup table in weight form, and the generalization we measure is within an interface, not across interfaces.

1 Introduction

Software is a state machine: a state, a set of actions a user can take, a transition function, and properties that must hold before and after every step. We ship it as code. This paper asks what it takes to ship it as a network instead—not a model that *writes* the program, but a network that *is* the program, in the plain sense that it takes the interface's state and a user action and returns the next state, and is held to a specification the way code is held to types and tests.

Preprint.

Three things make the question worth asking now. First, a behavior specification is a smaller and more checkable artifact than an implementation, and the part of it that is explicit can be *compiled* into weights rather than learned, so the network is exact where the spec is explicit and learning is spent only on what the spec can show but not write (ink-style holes [siliconjungle, 2026]) and on cells it never saw. Second, a network's runtime is one function—a forward pass and an argmin—that runs identically on a GPU trainer, a CPU worker, an edge isolate, or a browser tab, and an update to it is a sparse weight patch, not a deployment. Third, recent work compiles exact computation into the weights of language models [Agrawal, 2026, Vixhal, 2026, Velez-Ginorio et al., 2025a], which suggests a future in which an interface compiled this way is a component other networks can hold.

NERVE is the smallest system we could build that takes this seriously end to end. The input is a `.nerve` spec (Section 2), written by hand or produced from code, prose, or a screenshot by an agent that iterates against the spec checker. The output is a network whose energy $E(s, a, s')$ over candidate next states s' is a sum of three terms (Sections 3 and 4): a *compiled circuit*, exact by construction on every transition the spec writes down and constant elsewhere; a *JEPA* term [LeCun, 2022, Assran et al., 2023], the squared prediction error of an action-conditioned predictor in representation space; and a *factored per-field* term whose shift kernel is translation-equivariant along ordinal fields. Inference is $\arg\min_{s'} E$ over the states a gesture may lead to (the *futures* of declarative dragging [Horowitz et al., 2026]), or gradient descent on E for continuous fields. The network that ships is BF16; a Cloudflare Worker, Cloudflare's open-source `workerd`, and a static browser bundle execute the identical energy, and continued training reaches them as sparse patches that are verified bit-identical by digest (Section 7).

The repository this work lives in began as the end-to-end negotiator of Lewis et al. [2017]: one network, one task, supervised then self-play. We keep a finite version of that negotiation game as one of our specs and change the question from *can a network learn to negotiate* to *can a network be held to the rules of the game*.

Contributions. (1) A behavior-spec language with a mechanically derived reference interpreter and self-check, and an intake path from code, prose, or screenshots (Section 2). (2) An exact compiler from finite specs to ReLU weights, with a proof that the compiled energy equals the normalized Hamming distance to the true successor and a condition under which it dominates any learned term (Section 3). (3) An energy model that combines JEPA prediction error with a factored per-field head whose shift kernel is translation-equivariant along ordinal fields, and ablations that say where the head matters (shift rules, large hold-out) and where it does not (Sections 4 and 6). (4) An evaluation protocol for networks-as-programs: deterministic hold-out of (state, action) cells, learned-only scoring with the circuit disabled, violation rate against the spec, scores taken from the BF16 weights that ship, and runtime parity over HTTP (Section 5). (5) Results on eight specs, three of them from screenshots; ablations and seed variance; continuous inference; sparse patching; and a case study of an evolving agent team (Sections 6 and 7). Everything in this paper is one branch of one public repository and reproduces on a two-core CPU; the negotiation network trains in under five minutes.

2 Behavior specifications

A spec (Figure 1, left) declares typed *fields* $f_1, \ldots, f_n$ with domains $D_1, \ldots, D_n$ (bools, enums, bounded ints, floats with or without a step grid), *actions* with typed parameters, a guarded pure *transition* per action, *invariants* over states, *ensures* clauses over (old state, new state, parameters), *futures*, and *learn* holes.

Definition 1 (Finite spec)**.** A spec is *finite* when every field domain is finite (a float with a step grid counts) and no action has a continuous parameter. Its state space is $\mathcal{S} = \prod_i D_i$ and its action set $\mathcal{A}$ is the set of concrete action instances, one per combination of discrete parameter values. The transition function $T : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$ applies the action's update expressions when its guard holds and is the identity otherwise. For a hole action, T is the hole's *fallback*.

Expressions are evaluated by a whitelisted AST walker (arithmetic, comparisons, boolean operators, conditionals, `old.`f, and a fixed set of builtins); nothing is `eval`'d. The *oracle* is the interpreter of T together with a checker that returns, for a step (s, a, s'), the list of properties it breaks: a field out of type, an invariant false at s', or an ensures clause false. The oracle is the only "software" in the traditional sense, and it is derived mechanically from the spec, never written per interface.

```
nerve: 1
name: counter
state:
  count: {type: int, range: [0, 9]}
  armed: {type: bool}
actions:
  inc: {}
  dec: {}
  arm: {}
  set: {params: {to: {type: int, range: [0,
      9]}}}
  nudge: {}      # behaviour given only by
      examples
transitions:
  inc: { when: "armed", count: "min(count␣+␣1,␣
      9)" }
  dec: { when: "armed", count: "max(count␣-␣1,␣
      0)" }
  arm: { armed: "not␣armed" }
  set: { count: "to" }
invariants: ["0␣<=␣count␣<=␣9"]
ensures:
  - { after: inc, holds: "count␣>=␣old.count" }
  - { after: set, holds: "count␣==␣to" }
futures:
  drag_dial: { kind: closest, over: count,
               values: [0,1,2,3,4,5,6,7,8,9] }
learn:
  - name: nudge_towards_middle
    action: nudge
    fallback: { count: "count" }
    examples:
      - { state: {count: 0, armed: true},
          next:  {count: 2, armed: true} }
      # ... four more
initial: {count: 0, armed: true}
```

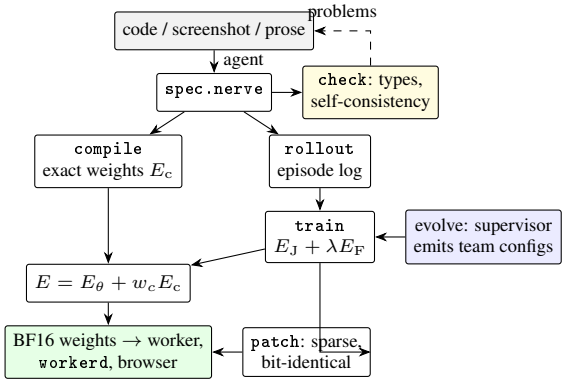


Figure 1: Left: the counter spec. Right: the pipeline. The spec is the only source of truth; the oracle, the circuit, the training data and the evaluation are all derived from it.

Self-check. Before anything is trained, `nerve check` runs the spec against itself: for every $s \in \mathcal{S}$ satisfying the invariants and every $a \in \mathcal{A}$ it asserts that $T(s, a)$ satisfies the invariants and the ensures clauses (for non-finite specs, over 2,000 random samples). A spec whose own transitions break its own properties never reaches training. The check caught a real error while we were writing the slider spec (an ensures clause that was false whenever the guard failed), and it is the loop condition for agent intake.

Futures and holes. Declarative dragging [Horowitz et al., 2026] describes a drag by the set of model states it may lead to (`closest`, `between`, `vary`). A `futures` block declares this set per gesture, and energy-based inference is literally scoring those candidates and taking the minimum. The full state space is the conservative default and is what we evaluate against. A `learn` hole, after ink's learnable holes with an `ensures ... else` fallback [siliconjungle, 2026], names behavior the spec can only show: an action, a safe fallback transition, and examples. The oracle knows only the fallback; the examples are ground truth the spec cannot express, and they enter training as positives (oversampled $20\times$) while the compiled circuit is made silent on that action.

Intake. Code, prose, or a screenshot is turned into a spec by an agent (Claude Code, or DeepSeek Harness) given the grammar and one of three prompts; the harness runs `nerve check` and sends back problems for up to three rounds. Three of the eight specs in this paper came from screenshots this way (Section 6).

3 Compiling the explicit part into weights

Let $\mathbf{x}(s) = [\text{onehot}(s_1); \ldots; \text{onehot}(s_n); \psi(s)] \in \mathbb{R}^{d_s}$ be the state features, where ψ collects the ordinal features of Section 4 (the circuit ignores them), and let $\mathbf{e}_a \in \mathbb{R}^{|\mathcal{A}|}$ be the action one-hot. For a finite spec with $H = |\mathcal{S}|\,|\mathcal{A}|$ we build a two-layer ReLU network with one hidden unit per pair

$k = (s_k, a_k)$:

$$h_k(s,a) = \mathrm{relu}\big(\langle \mathbf{x}(s), \mathbf{w}^s_k \rangle + \langle \mathbf{e}_a, \mathbf{w}^a_k \rangle - n\big), \qquad \mathbf{y}(s,a) = \textstyle\sum_k h_k(s,a)\, \mathbf{w}^o_k, \tag{1}$$

with $\mathbf{w}^s_k$ the one-hot block of $\mathbf{x}(s_k)$, $\mathbf{w}^a_k = \mathbf{e}_{a_k}$, and $\mathbf{w}^o_k$ the one-hot block of $\mathbf{x}(T(s_k, a_k))$, or $\mathbf{0}$ when a_k is a hole action. The circuit's energy of a candidate successor is

$$E_{\mathrm{c}}(s, a, s') = 1 - \tfrac{1}{n} \langle \mathbf{x}(s'), \mathbf{y}(s,a) \rangle. \tag{2}$$

Theorem 1 (The compiled energy is a Hamming distance)**.** *For every finite spec, every $s \in \mathcal{S}$, every non-hole $a \in \mathcal{A}$ and every $s' \in \mathcal{S}$, $E_{\mathrm{c}}(s,a,s') = d_{\mathrm{H}}\big(s', T(s,a)\big)/n$, where d_{H} counts the fields on which two states differ. In particular E_{c} is 0 exactly at the true successor and at least $1/n$ everywhere else. For a hole action, $E_{\mathrm{c}}(s,a,\cdot) \equiv 1$.*

Proof. Fix (s,a). The pre-activation of unit k is $\sum_i \mathbb{1}[s_i = s_{k,i}] + \mathbb{1}[a = a_k] - n$. If $s = s_k$ and $a = a_k$ it equals 1. If $s \neq s_k$, at least one field differs, so the sum is at most $(n-1) + 1 - n = 0$; if $s = s_k$ but $a \neq a_k$ it is $n + 0 - n = 0$. Hence exactly one unit fires, with value 1, and $\mathbf{y}(s,a) = \mathbf{w}^o_{(s,a)}$, the one-hot block of $T(s,a)$. Then $\langle \mathbf{x}(s'), \mathbf{y} \rangle = \sum_i \mathbb{1}[s'_i = T(s,a)_i] = n - d_{\mathrm{H}}(s', T(s,a))$, which gives the claim. For a hole action $\mathbf{w}^o = \mathbf{0}$, so $\langle \mathbf{x}(s'), \mathbf{y} \rangle = 0$ and $E_{\mathrm{c}} \equiv 1$. □

Corollary 1 (When the circuit decides)**.** *Let $E = E_\theta + w_c E_{\mathrm{c}}$ for any learned term E_θ. If for a given (s,a) the learned term satisfies $E_\theta(s,a,T(s,a)) - \min_{s'} E_\theta(s,a,s') < w_c/n$, then $\arg\min_{s'} E(s,a,s') = T(s,a)$. In particular, whenever the learned term's spread $\max_{s'} E_\theta - \min_{s'} E_\theta$ is below w_c/n, the circuit decides every non-hole transition regardless of what was learned; on hole actions the circuit contributes a constant and the learned term decides.*

Remark 1. The circuit is a lookup table in weight form: $H = |\mathcal{S}||\mathcal{A}|$ hidden units and $H(2d_s + |\mathcal{A}|)$ computed parameters (280 units and 15,680 parameters for the counter, 2,376 and 225,720 for negotiation). Its value is not compression but that it lives in the same function space as the learned energy, sums with it, and is exact there. We compile only when $H \leq 2 \times 10^5$; larger interfaces are carried by the learned model alone. The weight matters. With a fixed $w_c = 1000$ and $n \leq 5$ the bound is $w_c/n \geq 200$, and on the trained negotiation network the learned term's spread over candidates reaches 1665: the circuit was being outvoted in principle and agreed with the learned term only because the learned term was right. We therefore set the weight *per query*, $w_c = \max(1000,\, n \cdot \mathrm{spread} + 1)$, in the Python runtime and in the Worker alike, which makes Corollary 1 unconditional; a test trains nothing, multiplies the learned weights by 50, and checks that every compiled transition of every finite spec still matches the oracle while hole actions are still left to the learned term.

4 The learned energy

Features. Finite fields are one-hot. An *ordinal* field (an int or gridded float with more than two values) additionally contributes $\psi(u) = [u, \sin 2\pi k u, \cos 2\pi k u]_{k=1..4}$ of its normalized value $u \in [0,1]$, so that "same value" and "+1" are compositional in feature space. Continuous fields contribute $\psi(u)$ only, differentiably. Actions are one-hot over instances; a continuous parameter enters through a linear map added to the action embedding.

JEPA term. An encoder $f_\theta : \mathbb{R}^{d_s} \to \mathbb{R}^d$ ($d = 64$; three-layer GELU MLPs throughout), an action embedding $\mathbf{e}_a \mapsto \mathbf{z}_a \in \mathbb{R}^d$, and a predictor g_θ give

$$\hat{\mathbf{z}}' = g_\theta\big(f_\theta(\mathbf{x}(s)), \mathbf{z}_a\big), \qquad E_{\mathrm{J}}(s,a,s') = \big\|\hat{\mathbf{z}}' - \bar{f}(\mathbf{x}(s'))\big\|_2^2, \tag{3}$$

where $\bar{f}$ is an exponential moving average of f_θ (momentum 0.99) with no gradient, as in BYOL and I-JEPA [Grill et al., 2020, Assran et al., 2023]. The network never reconstructs pixels or DOM; it predicts in representation space, which is the right level for *what the interface does* as opposed to what it looks like. Rendering is a separate decoder and out of scope here.

Factored term. For each discrete field f with domain size n_f and current index i_f, let $\mathbf{x}_{-f}$ be the state features with f's own one-hot and ordinal features masked to zero. Two small MLPs

H_f, G_f read $[\mathbf{z}_a; \mathbf{x}_{-f}]$ and produce absolute logits $H_f \in \mathbb{R}^{n_f}$ and, for ordinal fields, shift logits $G_f \in \mathbb{R}^{2n_f-1}$ indexed by offset. The logit of next value j is

$$\ell_f(j \mid s, a) = H_f(\mathbf{z}_a, \mathbf{x}_{-f})[j] + \mathbb{1}[f \text{ ordinal}]\, G_f(\mathbf{z}_a, \mathbf{x}_{-f})[\, j - i_f + n_f - 1\,] + \epsilon\, O_f(\hat{\mathbf{z}}')[j], \quad (4)$$

with $\epsilon = 0.1$ a small own-value term for genuine exceptions, and $E_{\mathrm{F}}(s, a, s') = -\sum_f \log \mathrm{softmax}(\ell_f)[s'_f]$.

Proposition 1 (Shift equivariance along ordinal fields)**.** *Fix a and all fields other than f. Write $\ell_i^{\mathrm{shift}}(j) = G_f(\mathbf{z}_a, \mathbf{x}_{-f})[j - i + n_f - 1]$ for the shift contribution when the current value has index i. Then $\ell_{i+\delta}^{\mathrm{shift}}(j+\delta) = \ell_i^{\mathrm{shift}}(j)$ for all i, j, δ with both indices in range, and $H_f(\mathbf{z}_a, \mathbf{x}_{-f})$ does not depend on i at all.*

Proof. G_f and H_f receive $\mathbf{x}_{-f}$, which by construction carries no information about f's value, so they are constant in i; the shift index $j - i$ is invariant under $(i, j) \mapsto (i + \delta, j + \delta)$. □

So a rule such as `count: "min(count + 1, 9)"` under guard `armed`, once learned at $i = 3$, is the same function at $i = 8$; what the head *can* condition on is the action and every other field, which is exactly what guards refer to. Only the ϵ-weighted own-value term and the domain boundary break the symmetry, and the boundary is where the examples differ anyway (`min`, `max`, `clamp`). Section 6 shows where this matters: on the specs whose rules are shifts, increasingly as more cells are held out, and not at all on the negotiation game.

Total energy and inference. $E = E_{\mathrm{J}} + \lambda_F E_{\mathrm{F}} + w_c E_{\mathrm{c}}$ with $\lambda_F = 10$ and w_c set per query as in Section 3. For finite specs inference is $\arg\min$ over all states (or the declared futures). For a spec with continuous fields, candidates are the discrete combinations times a handful of seeds per continuous field (the current value, the action's parameter, 0, $\frac{1}{2}$, 1, and six random points); each seed is descended on E_{J} with Adam (60 steps, step size 0.05, clamped to the range) and the global minimum wins—dragging's `vary` as gradient descent on an energy.

Training. A run is a pure function of (spec, log, config, seed). The log is an append-only JSONL file written by policies that operate the oracle: *exhaustive* (every (state, action) pair once, for finite specs under 2×10^5 pairs), *coverage*, *random*, and *adversary* (prefers boundary-hugging actions). Every positive comes with $k = 3$ spec-derived negatives: candidate successors that break an invariant or an ensures clause, or are plainly wrong. The loss on a batch is

$$\mathcal{L} = \underbrace{\mathrm{CE}\big(-E_{\mathrm{J}}(s, a, \cdot)/\tau,\ T(s, a)\big)}_{\text{InfoNCE over all candidates}} + 0.1\, E_{\mathrm{J}}(s, a, T(s, a)) + \underbrace{\big[m + \bar{E}_+ - E_{\mathrm{J}}(s, a, s'_-)\big]_+}_{\text{hinge on violating futures}} + 0.05\, \mathcal{L}_{\mathrm{var}} + \mathcal{L}_{\mathrm{ord}} + \lambda_F\, E_{\mathrm{F}}(s, a, T(s, a)), \quad (5)$$

with $\tau = 0.1$, margin $m = 1$, $\mathcal{L}_{\mathrm{var}}$ the VICReg variance hinge [Bardes et al., 2022] (skipped on a batch of one, which otherwise produces a NaN that silently zeroes a run), and $\mathcal{L}_{\mathrm{ord}}$ a linear read-out of ordinal values from both $f_\theta(\mathbf{x}(s))$ and $\hat{\mathbf{z}}'$. AdamW with weight decay 0.05—the grokking lever [Power et al., 2022, Nanda et al., 2023]—learning rate 3×10^{-3} with cosine decay, batch 128, 300 epochs. Held-out cells (Section 5) are excluded from positives and negatives alike.

5 Measuring a network as a program

A program is right or wrong on every input, so we report exact match, not likelihood. Let P be the set of (valid state, non-hole action) pairs. A salted BLAKE2b hash of the pair assigns 15% of P to a *held-out* set, so the split is a property of the spec and the salt, not of the run; training never sees those cells, and tiny specs with fewer than 20 pairs get no hold-out at all (toggle). We report: *held-out exact*, the fraction of held-out cells where $\arg\min E$ equals T; *all exact*, the same over P; *violation rate*, the fraction of predicted successors over P that break an invariant or ensures clause—a wrong but legal answer is a bug, an illegal one is a safety failure; *learned-only exact*, all-exact with the circuit disabled, so the network cannot hide behind the compiler; and *hole accuracy* on the examples of each `learn` hole. The official score is 0.4 held-out $+ 0.2$ all $+ 0.2\,(1 - \text{violations}) + 0.2$ learned-only, blended $80/20$ with hole accuracy when the spec has holes; it is the single number the evolving team optimizes. Two rules matter more than the weights: every number is computed from the BF16

Table 1: Learned-only conformance (compiled circuit disabled; 15% of (state, action) cells held out of training). **Top:** runs from the repository record, one Hugging Face `a10g-small` job each, 300 epochs. **Bottom:** the four hand-written finite specs re-run for this paper on a two-core CPU with the same configuration and three seeds; we give the held-out count and the per-seed values. Toggle has two cells and no hold-out set.

spec	source	$\lvert\mathcal{S}\rvert \times \lvert\mathcal{A}\rvert$	held-out	held-out exact	all exact	viol.
repository record (GPU, one seed)						
toggle	hand	2×1	–	1.000	1.000	0
counter (with hole)	hand	20×14	34	1.000	1.000	0
slider (gridded)	hand	50×29	213	1.000	1.000	0
negotiate	hand	216×11	367	1.000	1.000	0
push-notifications toggle	screenshot	84×27		1.000	1.000	0
guest stepper	screenshot	8×3		1.000	1.000	0
volume slider	screenshot	404×103		1.000	1.000	0
slider (continuous)	hand	$2 \times \infty$	1,000	0.998 within 1% of range		0.002
re-run for this paper (CPU, seeds 0/1/2)						
toggle	hand	2×1	–	1.000/1.000/1.000	1.000/1.000/1.000	0/0/0
counter (with hole)	hand	20×14	34	0.971/0.971/1.000	0.996/0.996/1.000	0/0/0
slider (gridded)	hand	50×29	213	1.000/1.000/0.995	1.000/1.000/0.999	0/0/0
negotiate	hand	216×11	367	1.000/1.000/1.000	1.000/1.000/1.000	0/0/0

checkpoint that ships, never from the FP32 training copy, and the Worker that serves the network must agree with Python on every transition over HTTP before a number is reported. Training and evaluation share one hold-out function; an earlier version in which they disagreed for tiny specs reported held-out cells as seen, and we count that bug, with the NaN above, as having mattered more than any modelling idea.

6 Results

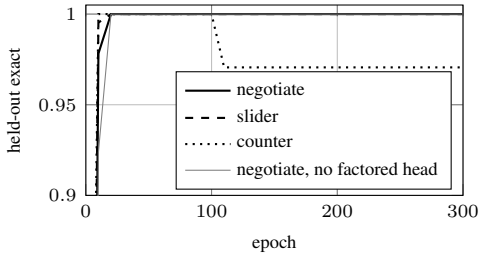
Specs. Five hand-written specs (toggle; the counter of Figure 1 with a `learn` hole; a gridded slider; the same slider with a continuous `value` and `to`; and a finite Deal-or-No-Deal in which two agents split two books and two hats over at most three rounds, 216 states $\times$ 11 action instances) and three specs produced from screenshots of real controls by the intake agent (a push-notifications toggle with a draggable knob, 84×27; a guest-count stepper, 8×3; a volume slider with mute, 404×103). The screenshot specs went image $\rightarrow$ spec (one or two rounds of `nerve check` feedback) $\rightarrow$ exhaustive log $\rightarrow$ network with no hand edits.

Conformance. Table 1 gives the repository's numbers and our re-runs. On the CPU re-runs, counter (with hole) misses 1, 1, 0 of 34 held-out cells across seeds 0–2; slider (gridded) misses 0, 0, 1 of 213 held-out cells across seeds 0–2; negotiate is exact on all 367 held-out cells in all 3 seeds; in total 6 of 9 (spec, seed) runs are exact on every held-out cell, and no run produces a spec violation. With the circuit enabled, all-exact is 1.000 on every finite spec by Theorem 1, and the hole examples of the counter are learned (1.00/1.00/1.00 on the five examples, seeds 0–2). The continuous slider, evaluated at 1,000 random (state, action) pairs with inference by gradient descent, reaches 0.998 within 1% of the range in the repository's GPU run; our CPU re-run with the same training configuration (200 epochs, $d = 128$) but the default 40-episode log reaches only 0.915, with 8% of predictions breaking the spec's clamp or guard, and since that row's episode log was not committed we cannot say whether data or hardware accounts for the gap. (The gridded slider had reached 0.986 before the factored head and ordinal features were added.)

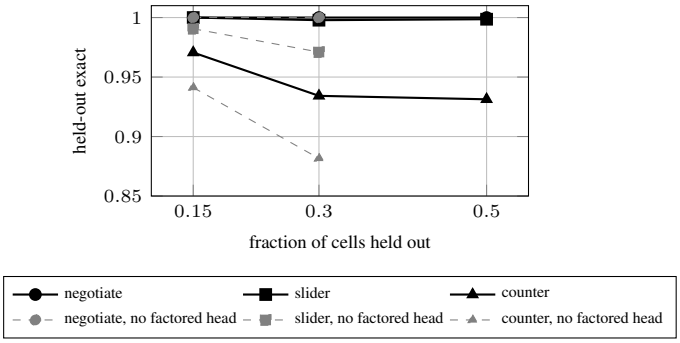
What does the work. Table 2 removes the pieces one at a time at the standard 15% hold-out, and Figure 2b holds out 30% and 50% of the cells. Three things are visible. First, with exhaustive logs, ordinal features and weight decay in place, the JEPA term alone is already close: negotiation is exact on every held-out cell without the factored head, without the ordinal read-out, and with weight decay 10^{-4}, and it stays exact when half of its 2,376 cells are held out—the rules of the game (`propose` assigns the parameters, flips the turn and increments the round; `accept` and `walk` set the status under

Table 2: Ablations on the three hand-written specs with ordinal fields (learned-only, 15% held out, seed 0 unless noted): held-out exact match and cells. At this hold-out the JEPA term alone is nearly sufficient; the factored head recovers the last cells on the two shift-rule specs and is irrelevant on negotiation.

	negotiate	slider	counter
full model (JEPA + ordinal read-out + factored head)	1.000 (367/367)	1.000 (213/213)	0.971 (33/34)
without factored head	1.000 (367/367)	0.991 (211/213)	0.941 (32/34)
without factored head and ordinal read-out	1.000 (367/367)	0.991 (211/213)	0.971 (33/34)
weight decay 10^{-4} (default 0.05)	1.000 (367/367)	1.000 (213/213)	0.971 (33/34)
without factored head, seeds 0–2	1.000/1.000/1.000		



(a) Held-out exact match during training (axis starts at 0.9); all three specs are exact by epoch 20.



(b) Held-out exact match as the held-out fraction grows (learned-only, seed 0).

Figure 2: Generalization inside an interface (learned-only, seed 0). Negotiation is recovered from half of its cells with or without the factored head; the shift-rule specs need it as more is held out; the counter loses one held-out cell late in training.

a guard) are recovered from the other half. Second, where a rule is a shift along an ordinal field the factored head is what carries the held-out cells, and increasingly so as fewer cells are seen: the slider goes from 211 to 213 of 213 at 15% and from 436 to 448 of 449 at 30%; the counter from 32 to 33 of 34 and from 67 to 71 of 76. Third, the counter is the hard case: 20 states, a guard (`armed`) that silences two actions, a hole, and range boundaries two cells from the middle; at 50% hold-out it reaches 122 of 131 with one predicted successor breaking an ensures clause, the only violation in the sweep. Figure 2a shows that generalization is not slow: every spec is exact on its held-out cells by epoch 20; the counter's single miss on seed 0 appears at epoch 110, after a hundred epochs of being right, which argues for a validation split separate from the held-out test cells rather than for more training.

Margins. On the negotiation network the energy gap between the true successor and the runner-up, learned-only, is at least 74 on held-out cells (median 93) and at least 75 on seen cells: held-out cells are not decided by a hair. The gap is in units dominated by $\lambda_F E_{\mathrm{F}}$, i.e. roughly $10\times$ a log-probability ratio.

Cost. Every network here has between 10^5 and 2×10^5 learned parameters and trains in 34 s to 9 min on an `a10g-small`, or 5.3 min for negotiation on two CPU cores. Inference is one forward pass plus the target encodings of the $|\mathcal{S}|$ candidates (cached in the Worker); a step on the 216-candidate negotiation network takes about 1.4 ms in Python on one CPU core and 0.6 ms on the counter.

7 Shipping it

BF16 and the edge. The checkpoint is BF16. `nerve export` writes the weights, the compiled circuit, the state vectors and the feature layout to one JSON document; `edge/src/nn.ts` (under

200 lines) is the entire runtime: GELU MLPs, the factored head, the circuit, and an argmin. The same bundle runs as a Cloudflare Worker with Durable Object sessions and R2 weights, locally on `workerd` with disk-backed weights, and as a static page in which the network runs in the browser (`docs/play`, all seven finite networks). The test suite trains small models, starts `workerd` on a spare port, and asserts that Python and the Worker agree on the state of every transition and on its energy to within 0.05 over HTTP; the weights agree bit for bit by digest, the energies only to tolerance because the Worker's GELU is the tanh approximation and PyTorch's is exact.

Sparse patches. Continued training at the learning rates used for RL post-training changes few BF16 bits; Miahi and Belilovsky [2026] measure $\sim 99\%$ update sparsity on 0.5–7B models and propose PULSE: send only changed indices and their new absolute values, verify a digest. We implement the same format (varint index deltas, BF16 values, zstd, SHA-256 of the full post-patch checkpoint) in Python and in the Worker; a patch is applied as a memory copy and rejected—weights restored—if the digest does not match, and a corrupted frame is likewise refused. On the negotiation network (140k parameters) one further epoch at 3×10^{-6} changes 7.8% of BF16 values and ships as a 31 kB patch against a 280 kB checkpoint; at the training learning rate (3×10^{-3}, five epochs) 99% of values change and the patch is $1.2\times$ smaller than the checkpoint only through compression (Table 5). Our networks are three to five orders of magnitude smaller than the models PULSE was designed for, so the ratios are smaller too; the property we rely on is bit-identity, not compression.

An evolving team built the last one. Cao et al. [2026] describe an Evolving Agent Team: a supervisor agent that never writes the solution and each iteration reflects on the last team's result and emits a team configuration (roles, prompts, phases, ownership); the team, made of coding agents, does the work; a fixed evaluation scores it. `nerve evolve` is that loop over this pipeline: roles own `spec`, `rollout`, `adversary`, `train` or `eval`; a spec role may add missing guards and properties to a working copy but every edit must pass `nerve check` or is reverted; only the harness trains. An offline rule-based reflector makes the loop reproducible with no model at all (and is what the tests run). With Claude Code as supervisor and role agents, and one GPU job per iteration, the negotiation spec ran 11 iterations for $\$14.23$, reached 1.0 on every metric, invented four roles that did not exist at iteration 0 (`split_prober`, `spec_verifier`, `margin_auditor`, `margin_checker`), and, once the score was saturated, restructured the team to cut its own cost per iteration. The run's record (every team configuration, evaluation and role report) is part of the artifact; see Section 9 on its provenance.

8 Related work

Programs in weights. That finite automata can be constructed in networks is classical [Omlin and Giles, 1996, Siegelmann and Sontag, 1995]; Tracr compiles RASP programs into transformer weights [Lindner et al., 2023], Cajal compiles a typed linear language to linear and recurrent neurons with a correctness proof [Velez-Ginorio et al., 2025a,b], ink's `emit-circuit` lowers functions over finite types to exact ReLU nets [siliconjungle, 2026], and Qwenputer and qwenGPU compile a CPU and a GPU into layers in front of a pretrained language model [Agrawal, 2026, Vixhal, 2026]. Our circuit is the simplest member of this family—a table—and its contribution is the energy framing (Theorem 1) that lets it sum with a learned term and be silent where the spec is.

Energy-based inference and JEPA. Scoring candidates and taking the minimum is energy-based learning [LeCun et al., 2006]; implicit behavioral cloning trains energy policies with InfoNCE over sampled actions and infers by argmin or descent [Florence et al., 2021]. JEPA predicts in representation space with an EMA target [LeCun, 2022, Assran et al., 2023, Grill et al., 2020]; latent world models plan by descending prediction error [Zhou et al., 2024, Sobal et al., 2025]. We use the prediction error as the energy of a program and add a factored equivariant term because, for a state machine, the field-wise structure is known.

Neural interfaces and games. NeuralOS [Rivard et al., 2026], GameNGen [Valevski et al., 2024] and Genie [Bruce et al., 2024] learn interactive systems as frame predictors from recordings. They answer what the screen will look like; we answer what the state will be, hold the answer to a spec, and treat rendering as a separate decoder. The end-to-end negotiator [Lewis et al., 2017] learned a dialogue policy; our negotiation spec is a referee for such policies.

Specifications and holes. Typed holes with live semantics [Omar et al., 2019], program sketching [Solar-Lezama, 2008] and neurosymbolic programs with neural holes [Gaunt et al., 2016, Chaudhuri et al., 2021] are the ancestors of `learn`; declarative dragging [Horowitz et al., 2026] is the ancestor of `futures`; TLA$^+$ and Alloy [Lamport, 2002, Jackson, 2012] are the ancestors of the self-check. Weight update sparsity and PULSE are from Miahi and Belilovsky [2026]; the evolving-team loop is from Cao et al. [2026]; grokking with weight decay from Power et al. [2022], Nanda et al. [2023].

9 Limitations, and what this is not

The interfaces are small. The largest finite spec has 404 states and 103 action instances; the circuit is a table, and "learned-only 1.0" on $41{,}612$ cells is generalization *within* one interface from 85% of its cells to the rest, not generalization across interfaces. Nothing here transfers from one spec to another; a shared backbone is future work. *The oracle is code.* The spec is executed by an interpreter derived from it; "software without code" means the shipped artifact is weights, not that no program exists. The interpreter is small, generic, and never written per interface, which is the point. *The circuit's guarantee was conditional until this paper.* Corollary 1 needs the learned term's spread below w_c/n and the trained networks exceeded it; the per-query weight of Section 3 was added while writing, and the runs in Tables 1 and 2 predate it (they are learned-only and unaffected). *Rendering is out of scope.* The network is the behavior; what the interface looks like is a decoder we do not learn. *Seeds and splits.* Our re-runs use three seeds and one hash salt; the held-out *set* is fixed across seeds, so the seed variance we report covers initialization and shuffling, not split choice. Worse, the evolving team optimizes the official score, which is computed on those same held-out cells, so in an evolution run they are a validation set and the reported number is optimistic; the artifact now takes a salt so that a second, sealed split can be the test. *The evolution record.* The 11-iteration run is recorded in the repository's working tree under `docs/runs/`, a path that the branch's `.gitignore` excluded at the time of writing; the record must be committed before the claim can be checked by a reader, and we treat it as a case study rather than a result until then. *Negotiation is finite and silent.* The game has no language; it is the rules of the game held in weights, which is the first half of the original repository's task and not the second.

10 Conclusion

A small interactive program can be shipped as a network and held to its specification: compiled where the spec is explicit, trained where it is not, scored on the weights that ship, and executed by one function at the edge. Exhaustive coverage of the seen cells mattered as much as any modelling idea; the idea that mattered where coverage was thin was an equivariant factored energy that makes a shift rule the same function at every value of a field; the two bugs that mattered were a NaN and a split disagreement. The next steps are in the repository's roadmap: factored and verified circuits, hard refusal at the edge, a shared backbone across specs, and bringing the 2017 dataset back so that two language agents negotiate over a state the network also enforces. One of them deserves a sentence here. Dragology's drag operators each return, per pointer frame, a preview state, a drop state and a *gap*—the distance from the pointer to where the dragged element lands in that state—and `closest`, `between` and `vary` are argmin, convex relaxation and gradient descent on that gap [Horowitz et al., 2026]. The gap is an energy, so a drag is one minimization of $E_{\text{spec}} + \lambda\,\text{gap}$ over futures the spec derives rather than the author lists; the operator memory (stickiness, lock, far/near) is a small guarded state machine the spec language already has words for; and the shift kernel of Section 4 is the ordinal case of a rule—equivariance chosen by the field's type—whose list case (permutation equivariance) is exactly what Dragology's reordering demos need. The design is written out in the repository (`docs/dragology.md`).

Reproducibility. `scripts/demo.sh counter` runs the pipeline on a CPU in about a minute; the sweep script and raw results behind Tables 1 and 2 and Figure 2 are in `docs/paper/` and regenerate every number in this paper.

References

Divyansh Agrawal. Qwenputer: a computer, compiled into the weights of a language model. https://github.com/divagr18/qwenputer, 2026.

Mahmoud Assran, Quentin Duval, Ishan Misra, Piotr Bojanowski, Pascal Vincent, Michael Rabbat, Yann LeCun, and Nicolas Ballas. Self-supervised learning from images with a joint-embedding predictive architecture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023.

Adrien Bardes, Jean Ponce, and Yann LeCun. VICReg: Variance-invariance-covariance regularization for self-supervised learning. In *International Conference on Learning Representations (ICLR)*, 2022.

Jake Bruce, Michael Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, Yusuf Aytar, Sarah Bechtle, Feryal Behbahani, Stephanie Chan, Nicolas Heess, Lucy Gonzalez, Simon Osindero, Sherjil Ozair, Scott Reed, Jingwei Zhang, Konrad Zolna, Jeff Clune, Nando de Freitas, Satinder Singh, and Tim Rocktäschel. Genie: Generative interactive environments. In *International Conference on Machine Learning (ICML)*, 2024.

Shiyi Cao, Ziming Mao, Dacheng Li, Joseph E. Gonzalez, and Ion Stoica. Evolving agent teams, 2026. arXiv:2610.36848; to appear at NeurIPS 2026. Code: https://github.com/caoshiyi/EAT.

Swarat Chaudhuri, Kevin Ellis, Oleksandr Polozov, Rishabh Singh, Armando Solar-Lezama, and Yisong Yue. Neurosymbolic programming. *Foundations and Trends in Programming Languages*, 7(3), 2021.

Pete Florence, Corey Lynch, Andy Zeng, Oscar A. Ramirez, Ayzaan Wahid, Laura Downs, Adrian Wong, Johnny Lee, Igor Mordatch, and Jonathan Tompson. Implicit behavioral cloning. In *Conference on Robot Learning (CoRL)*, 2021.

Alexander L. Gaunt, Marc Brockschmidt, Rishabh Singh, Nate Kushman, Pushmeet Kohli, Jonathan Taylor, and Daniel Tarlow. TerpreT: A probabilistic programming language for program induction, 2016. arXiv:1608.04428.

Jean-Bastien Grill, Florian Strub, Florent Altché, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Avila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

Joshua Horowitz, Andrew Blinn, and Jeffrey Heer. Declarative dragging. In *Proceedings of the ACM Symposium on User Interface Software and Technology (UIST)*, 2026. Best Paper Award. Library: https://github.com/declarative-dragging/dragology.

Daniel Jackson. *Software Abstractions: Logic, Language, and Analysis*. MIT Press, revised edition, 2012.

Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

Yann LeCun. A path towards autonomous machine intelligence, 2022. OpenReview, version 0.9.2.

Yann LeCun, Sumit Chopra, Raia Hadsell, Marc'Aurelio Ranzato, and Fu Jie Huang. A tutorial on energy-based learning. In *Predicting Structured Data*. MIT Press, 2006.

Mike Lewis, Denis Yarats, Yann N. Dauphin, Devi Parikh, and Dhruv Batra. Deal or no deal? end-to-end learning for negotiation dialogues. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2017. arXiv:1706.05125.

David Lindner, János Kramár, Sebastian Farquhar, Matthew Rahtz, Thomas McGrath, and Vladimir Mikulik. Tracr: Compiled transformers as a laboratory for interpretability. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Erfan Miahi and Eugene Belilovsky. Understanding and exploiting weight update sparsity for communication-efficient distributed RL, 2026. arXiv:2602.03839.

Neel Nanda, Lawrence Chan, Tom Lieberum, Jess Smith, and Jacob Steinhardt. Progress measures for grokking via mechanistic interpretability. In *International Conference on Learning Representations (ICLR)*, 2023.

Cyrus Omar, Ian Voysey, Ravi Chugh, and Matthew A. Hammer. Live functional programming with typed holes. *Proceedings of the ACM on Programming Languages*, 3(POPL), 2019.

Christian W. Omlin and C. Lee Giles. Constructing deterministic finite-state automata in recurrent neural networks. *Journal of the ACM*, 43(6):937–972, 1996.

Alethea Power, Yuri Burda, Harri Edwards, Igor Babuschkin, and Vedant Misra. Grokking: Generalization beyond overfitting on small algorithmic datasets, 2022. arXiv:2201.02177.

Luke Rivard, Sun Sun, Hongyu Guo, Wenhu Chen, and Yuntian Deng. NeuralOS: Towards simulating operating systems via neural generative models. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2507.08800.

Hava T. Siegelmann and Eduardo D. Sontag. On the computational power of neural nets. *Journal of Computer and System Sciences*, 50(1):132–150, 1995.

siliconjungle. Ink: a small language with locally checked proofs and an extensible optimisation database. https://github.com/siliconjungle/ink-lang, 2026. Learnable holes, `ensures ... else` fallbacks, and `ink emit-circuit`.

Vlad Sobal, Wancong Zhang, Kyunghyun Cho, Randall Balestriero, Tim G. J. Rudner, and Yann LeCun. Learning from reward-free offline data: A case for planning with latent dynamics models, 2025. arXiv:2502.14819.

Armando Solar-Lezama. *Program Synthesis by Sketching*. PhD thesis, University of California, Berkeley, 2008.

Dani Valevski, Yaniv Leviathan, Moab Arar, and Shlomi Fruchter. Diffusion models are real-time game engines, 2024. arXiv:2408.14837.

Joey Velez-Ginorio, Nada Amin, Konrad Kording, and Steve Zdancewic. Compiling to linear neurons, 2025a. arXiv:2511.13769.

Joey Velez-Ginorio, Nada Amin, Konrad Kording, and Steve Zdancewic. Compiling to recurrent neurons, 2025b. arXiv:2511.14953.

Vixhal. qwenGPU: a SIMT GPU compiled into the weights of Qwen3.5-2B. https://github.com/TheVixhal/qwenGPU, 2026.

Gaoyue Zhou, Hengkai Pan, Yann LeCun, and Lerrel Pinto. DINO-WM: World models on pre-trained visual features enable zero-shot planning, 2024. arXiv:2411.04983.

A The spec language

A `.nerve` file is YAML with the following keys. Types: `{type: int, range: [lo, hi]}`, `{type: enum, values: [...]}`, `{type: bool}`, `{type: float, range: [lo, hi], step: s}` (gridded) or without `step` (continuous).

```
nerve: 1
name: <identifier>
state:        { <field>: <type>, ... }
actions:      { <action>: {} | {params: {<p>: <type>}}, ... }
transitions:  { <action>: { when: "<guard>", <field>: "<expr>", ... }, ... }
invariants:   [ "<expr over state>", ... ]
ensures:      [ { after: <action> | omitted, holds: "<expr over new state, params, old.*>" }, ... ]
futures:      { <gesture>: { kind: closest | between | vary, over: <field>, values: [...] } }
learn:        [ { name, action, fallback: {<field>: "<expr>"},
                  examples: [ { state: {...}, params: {...}, next: {...} } ] } ]
initial:      { <field>: <value> }
```

Expressions: literals, field names, parameter names, `old.`f, the operators `+ - * / // % **`, chained comparisons, `and/or/not`, `x if c else y`, and the builtins `min max abs clamp len int bool`. The parser rejects any other syntax node, any call that is not a builtin, and any attribute access that is not on `old`. YAML's on/off/yes/no are strings, not booleans, so enum values such as `[off, on]` survive. A hole example must give every field of `state` and `next` and, if the hole's action takes parameters, a `params` block; an example outside the declared types is a parse error.

The negotiation spec. Two agents split two books and two hats; a proposal is what the proposer keeps; at most three rounds; terminal states absorb.

```
nerve: 1
name: negotiate
state:
  turn:   {type: enum, values: [a, b]}
  books:  {type: int, range: [0, 2]}
  hats:   {type: int, range: [0, 2]}
  status: {type: enum, values: [open, deal, no_deal]}
  rounds: {type: int, range: [0, 3]}
actions:
  propose: {params: {b: {type: int, range: [0, 2]}, h: {type: int, range: [0, 2]}}}
  accept: {}
  walk: {}
transitions:
  propose: { when: "status␣==␣'open'␣and␣rounds␣<␣3",
             books: "b", hats: "h", turn: "'b'␣if␣turn␣==␣'a'␣else␣'a'", rounds: "rounds␣+␣1" }
  accept:  { when: "status␣==␣'open'␣and␣rounds␣>␣0", status: "'deal'" }
  walk:    { when: "status␣==␣'open'", status: "'no_deal'" }
invariants: ["0␣<=␣books␣<=␣2␣and␣0␣<=␣hats␣<=␣2", "rounds␣<=␣3"]
ensures:
  - { after: propose, holds: "turn␣!=␣old.turn␣or␣old.status␣!=␣'open'␣or␣old.rounds␣>=␣3" }
  - { after: accept,  holds: "status␣!=␣'open'␣or␣old.rounds␣==␣0" }
  - { holds: "status␣==␣old.status␣or␣old.status␣==␣'open'" }
initial: {turn: a, books: 0, hats: 0, status: open, rounds: 0}
```

B Encoding, sizes and hyper-parameters

Table 3: Per-spec sizes. d_s is the state feature width (one-hots plus 9 ordinal features per ordinal field), $|\mathcal{A}|$ the number of action instances, H the number of compiled hidden units, and "learned" the number of trainable parameters of the JEPA and factored head ($d = 64$, hidden 128).

spec	$\lvert\mathcal{S}\rvert$	$\lvert\mathcal{A}\rvert$	d_s	H	circuit params	learned params
toggle	2	1	2	2	10	96,196
counter	20	14	21	280	15,680	111,212
slider (gridded)	50	29	36	1,450	146,450	121,832
negotiate	216	11	42	2,376	225,720	140,018
push-notifications toggle	84	27	34	2,268	–	126,594[†]
guest stepper	8	3	17	24	–	102,961[†]
volume slider	404	103	114	41,612	–	188,338[†]

[†]Counted from the exported browser weights (includes the factored head's mask buffer); the specs themselves were not yet committed at the time of writing.

Training configuration for every finite run in this paper unless stated: 300 epochs, batch 128, AdamW (β defaults) with learning rate 3×10^{-3}, cosine decay to 5%, weight decay 0.05, EMA momentum 0.99, InfoNCE temperature $\tau = 0.1$, hinge margin 1, variance weight 0.05, ordinal read-out weight 1, factored weight $\lambda_F = 10$, own-value scale $\epsilon = 0.1$, hole oversampling 20×, hold-out 15%, circuit weight $w_c = 1000$. The continuous slider used 200 epochs and $d = 128$. The episode log for each spec is produced by `nerve rollout` with its default policies (exhaustive, coverage, random, adversary; 40 episodes of horizon 20 each; 3 negatives per positive). Rollout knobs are capped (400 episodes, horizon 60, 10 negatives) because an agent once asked for 26,000 negatives per step and produced a 2.5 GB log.

C Every run

Table 4 lists every training run behind Tables 1 and 2 and Figure 2, from the raw `results.jsonl` written by the sweep script. "held" is held-out exact match with the number of held-out cells, "all" is exact match over all valid cells, "viol" the violation rate, "holes" the hole-example accuracy where the spec has holes.

Table 4: All CPU runs for this paper (learned-only unless the tag says `circuit`).

tag	configuration	held	n_{held}	all	viol	holes	params	wall (s)
`toggle_main_s0`	s0	1.000	2	1.000	0	–	96,196	32
`counter_main_s0`	s0	0.971	34	0.996	0	1.00	111,212	51
`slider_main_s0`	s0	1.000	213	1.000	0	–	121,832	774
`negotiate_main_s0`	s0	1.000	367	1.000	0	–	140,018	322
`toggle_main_s1`	s1	1.000	2	1.000	0	–	96,196	32
`counter_main_s1`	s1	0.971	34	0.996	0	1.00	111,212	38
`slider_main_s1`	s1	1.000	213	1.000	0	–	121,832	57
`negotiate_main_s1`	s1	1.000	367	1.000	0	–	140,018	108
`toggle_main_s2`	s2	1.000	2	1.000	0	–	96,196	35
`counter_main_s2`	s2	1.000	34	1.000	0	1.00	111,212	38
`slider_main_s2`	s2	0.995	213	0.999	0	–	121,832	57
`negotiate_main_s2`	s2	1.000	367	1.000	0	–	140,018	340
`negotiate_nofact_s0`	s0, no fact	1.000	367	1.000	0	–	102,723	1267
`slider_nofact_s0`	s0, no fact	0.991	213	0.999	0	–	102,209	45
`counter_nofact_s0`	s0, no fact	0.941	34	0.992	0	1.00	97,409	31
`negotiate_purejepa_s0`	s0, no fact, no ord	1.000	367	1.000	0	–	102,723	62
`negotiate_wd1e-4_s0`	s0, wd 0.0001	1.000	367	1.000	0	–	140,018	103
`slider_purejepa_s0`	s0, no fact, no ord	0.991	213	0.999	0	–	102,209	46
`slider_wd1e-4_s0`	s0, wd 0.0001	1.000	213	1.000	0	–	121,832	60
`counter_purejepa_s0`	s0, no fact, no ord	0.971	34	0.996	0	1.00	97,409	28
`counter_wd1e-4_s0`	s0, wd 0.0001	0.971	34	0.996	0	1.00	111,212	36
`negotiate_hold0.3_s0`	s0, hold 0.3	1.000	741	1.000	0	–	140,018	83
`negotiate_hold0.5_s0`	s0, hold 0.5	1.000	1195	1.000	0	–	140,018	66
`negotiate_hold0.3_nofact_s0`	s0, hold 0.3, no fact	1.000	741	1.000	0	–	102,723	56
`slider_hold0.3_s0`	s0, hold 0.3	0.998	449	0.999	0	–	121,832	46
`slider_hold0.5_s0`	s0, hold 0.5	0.999	731	0.999	0	–	121,832	40
`slider_hold0.3_nofact_s0`	s0, hold 0.3, no fact	0.971	449	0.991	0	–	102,209	40
`counter_hold0.3_s0`	s0, hold 0.3	0.934	76	0.981	0	1.00	111,212	31
`counter_hold0.5_s0`	s0, hold 0.5	0.931	131	0.965	0.0038	0.80	111,212	23
`counter_hold0.3_nofact_s0`	s0, hold 0.3, no fact	0.882	76	0.965	0.0038	1.00	97,409	24
`negotiate_nofact_s1`	s1, no fact	1.000	367	1.000	0	–	102,723	64
`negotiate_nofact_s2`	s2, no fact	1.000	367	1.000	0	–	102,723	62
`counter_circuit_s0`	s0, circuit	1.000	34	1.000	0	1.00	111,212	40
`negotiate_circuit_s0`	s0, circuit	1.000	367	1.000	0	–	140,018	108
`slider_continuous_main_s0`	s0, d=128, circuit, 200 ep	0.915	1000	0.915	0.0810	–	145,156	75

D Sparse patches

Table 5: PULSE patches on the negotiation network (140,018 learned parameters; the full BF16 checkpoint is 280,456 bytes). Continued training from the seed-0 model on the same log for the given number of epochs and learning rate; sparsity is the fraction of BF16 values unchanged; every patch verified bit-identical by digest in Python and in the Worker's decoder.

learning rate	epochs	BF16 values changed	patch bytes	ratio
$3e-6$	1	7.81%	30,615	9.2×
$3e-6$	5	16.80%	58,700	4.8×
$3e-5$	1	24.37%	77,930	3.6×
$3e-5$	5	35.22%	107,192	2.6×
$3e-4$	1	54.85%	150,635	1.9×
$3e-4$	5	62.46%	170,247	1.6×
$3e-3$	1	97.55%	226,781	1.2×
$3e-3$	5	98.72%	227,333	1.2×

Patch format: `"NRVP"` | version | header length | JSON header (per-tensor: kind, shape, count, byte lengths; the SHA-256 of the full post-patch checkpoint over sorted tensor names and little-endian

BF16 bits) | zstd(body), where the body holds, per tensor, varint-encoded index deltas followed by the new BF16 values. Decoding is a memory copy; no floating-point arithmetic is performed, which is why chained patches cannot drift.

E The evolving team

The supervisor prompt asks for one JSON team configuration with `rationale`, `roles[{name, prompt, owns, knobs}]`, `phases[{name, roles, gate}]`, `train{}` and `rollout{}`. The default team is `spec_auditor` (owns `spec`; gate `spec.problems == 0`), `coverage_hunter` and `adversary` (data), `trainer` (knobs only; the harness trains), and `energy_auditor` (writes to the shared learnings). Role agents may run `nerve check|rollout|eval|step|compile` and a small set of shell tools; they may edit only the iteration directory and the episode log; a spec edit that fails `nerve check` is reverted and the author's specs are restored with `git checkout`. Agent-supplied knobs are coerced to the dataclass types and clamped to bounds (epochs 1–2000, d 8–512, learning rate 10^{-6}–0.05, hold-out 0–0.5, ...). The offline reflector encodes six rules of the form "learned-only < 0.9: more coverage and longer training", "held-out trails seen by > 0.02: wider d, more negatives", "holes < 1: add a `hole_tutor` role, lower the learning rate", "violations > 0: more negatives", and "plateau: halve the learning rate".

F Use of language models

The `nerve` branch and this manuscript were written by the author with Claude (Anthropic) acting as a coding and writing agent under the author's direction; the evolving-team experiment uses Claude Code as supervisor and role agents by design and the intake experiments use it as the vision agent. Every number reported from the repository record was re-derived by running the committed code where a CPU sufficed, and is marked as the repository's where it did not.

G NeurIPS paper checklist

1. **Claims.** The abstract and introduction claim exact-match conformance on small specified interfaces, a compilation theorem, an equivariance proposition, and system properties (BF16 scoring, runtime parity, bit-identical patches). Each is tied to a theorem, a table, or a test in the repository. [Yes]
2. **Limitations.** Section 9. [Yes]
3. **Theory.** Theorem 1, Corollary 1, and Proposition 1 state their assumptions and are proved in the text. [Yes]
4. **Experimental reproducibility.** Specs, logs, configurations, seeds and the sweep script are in the repository; every CPU run in Table 4 is reproducible by the listed command. The three screenshot specs and the evolution record were not committed at the time of writing and are flagged as such. [Partially; flagged]
5. **Open access to data and code.** Yes: one public branch; no external dataset.
6. **Experimental settings.** Appendix B. [Yes]
7. **Statistical significance.** Exact-match scores are reported per seed with held-out counts rather than as intervals: with 34–367 held-out cells and scores at or near 1.0 an interval is degenerate, and the honest statistic is the number of cells missed. [Yes]
8. **Compute.** Each GPU run is one `a10g-small` job of 34 s to 9 min; the full CPU sweep in Table 4 took 1.2 hours on two cores. Development compute, including the $14.23 evolution run, is disclosed in the text. [Yes]
9. **Code of ethics.** Conforms. [Yes]
10. **Broader impacts.** The work replaces small interface code with checked networks; the risks are those of any learned component in a user-facing system (silent errors), which the spec, the violation metric and the proposed hard refusal address. No dual-use concern is apparent. [Yes]
11. **Safeguards.** Not applicable (no released pretrained model with misuse potential).
12. **Licenses.** The repository descends from `facebookresearch/end-to-end-negotiator` and keeps its license; Dragology, ink, PULSE and EAT are cited and not redistributed. [Yes]

13. **New assets.** The spec language, pipeline and specs are released with documentation. [Yes]
14. **Crowdsourcing / human subjects.** None.
15. **IRB.** Not applicable.
16. **LLM usage.** Language-model agents are part of the method (intake; the evolving team) and are described in Sections 2 and 7; their use in writing is disclosed in Appendix F. [Yes]